AIR 시스템 분석 (Algebraic Intermediate Representation)

개요

AIR(Algebraic Intermediate Representation)는 Plonky3에서 계산을 다항식 제약조건으로 표현하는 핵심 추상화 계층입니다. AIR 시스템은 임의의 계산을 STARK 증명 시스템에서 검증 가능한 형태로 변환하는 역할을 담당하며, 다양한 암호학적 프리미티브(해시 함수, 서명 등)를 통합된 방식으로 처리할 수 있게 합니다.

이 문서에서는 AIR의 핵심 개념부터 시작하여 구체적인 구현 방식, 그리고 Blake3, Keccak, Poseidon2 등 주요 해시 함수의 AIR 구현을 비교 분석합니다.

AIR의 기본 개념

1. AIR란 무엇인가?

AIR는 계산을 **다항식 제약조건의 집합**으로 표현하는 방법입니다. 기본 아이디어는 다음과 같습니다:

1. **실행 추적(Execution Trace)**: 계산의 각 단계를 행렬의 행으로 표현
2. **제약조건(Constraints)**: 올바른 계산을 보장하는 다항식 방정식들
3. **검증**: 제약조건이 모든 행에서 만족되는지 확인

```
// AIR의 핵심 트레이트
pub trait Air<AB: AirBuilder>: BaseAir<AB::F> {
/// 모든 AIR 제약조건을 평가하는 함수
fn eval(&self, builder: &mut AB);
```

```
}
```

2. AirBuilder 인터페이스

`AirBuilder` 는 제약조건을 정의하고 평가하는 도구를 제공합니다:

```
pub trait AirBuilder: Sized {
type F: PrimeCharacteristicRing + Sync; // 기본 필드
type Expr: Algebra<Self::F>; // 표현식 타입
type Var: Into<Self::Expr>; // 변수 타입
type M: Matrix<Self::Var>; // 행렬 타입

// 추적 데이터 접근
fn main(&self) -> Self::M;

// 행 선택자
fn is_first_row(&self) -> Self::Expr;
fn is_last_row(&self) -> Self::Expr;
fn is_transition(&self) -> Self::Expr;

// 제약조건 정의
fn assert_zero<I: Into<Self::Expr>>(&mut self, x: I);
fn assert_eq<I1: Into<Self::Expr>, I2: Into<Self::Expr>>(&mut self, x: I1, y:
fn assert_bool<I: Into<Self::Expr>>(&mut self, x: I);
```

```
}
```

3. 조건부 제약조건

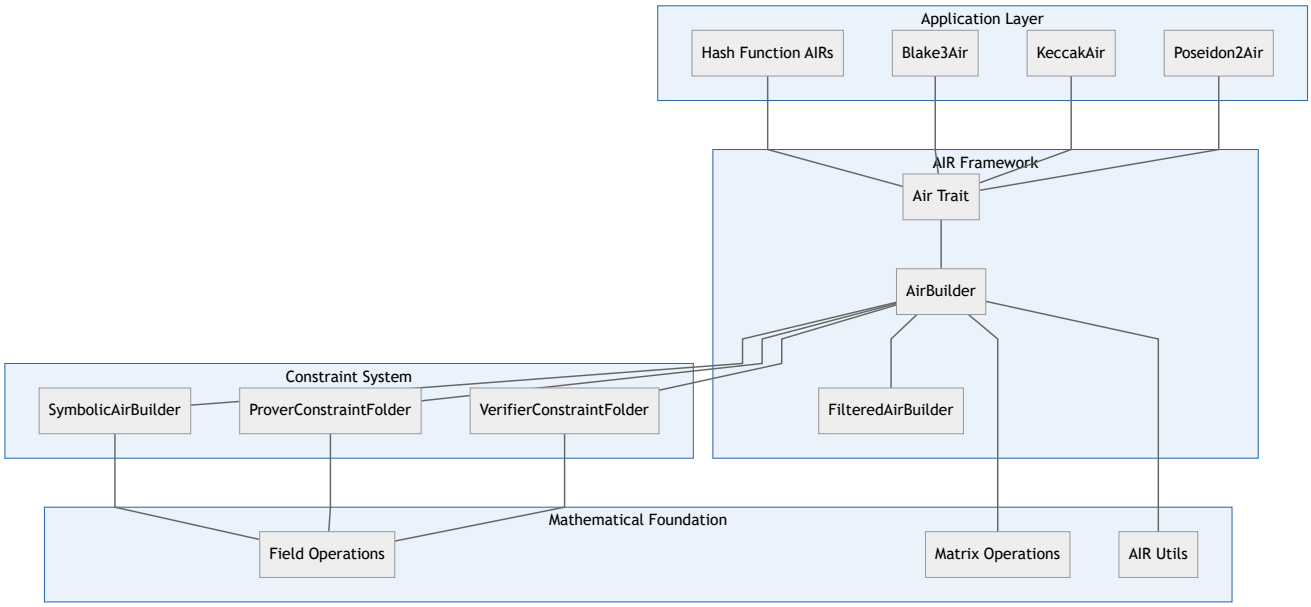
`FilteredAirBuilder` 를 통해 특정 조건에서만 적용되는 제약조건을 정의할 수 있습니다:

```
// 첫 번째 행에서만 적용되는 제약조건
builder.when_first_row().assert_eq(initial_state, expected_value);


// 전환 행에서만 적용되는 제약조건
builder.when_transition().assert_eq(next_state, computed_next_state);
```

AIR 구현 아키텍처

1. 계층 구조



2. 가상 컬럼 시스템

`VirtualPairCol` 을 통해 전처리된 데이터와 메인 추적 데이터를 조합할 수 있습니다:

```
pub struct VirtualPairCol<F: Field> {
column_weights: Vec<(PairCol, F)>,
constant: F,
}
```

```
pub enum PairCol {
Preprocessed(usize), // 전처리된 컬럼
Main(usize), // 메인 추적 컬럼
}
```

이를 통해 복잡한 선형 조합을 효율적으로 표현할 수 있습니다:

```
// 두 메인 컬럼의 차이
let diff = VirtualPairCol::diff_main(col_a, col_b);

// 여러 전처리 컬럼의 합
let sum = VirtualPairCol::sum_preprocessed(vec![0, 1, 2]);
```

해시 함수별 AIR 구현 분석

1. Blake3 AIR 구현

Blake3는 32비트 워드 기반의 해시 함수로, 복잡한 비트 연산과 모듈러 산술을 포함합니다.

주요 특징:

- **워드 표현**: 32비트 워드를 16비트 limb 2개로 분할
- **비트 연산**: XOR, 회전 연산을 비트 레벨에서 검증
- **모듈러 산술**: 2^32 모듈로 덧셈 검증

```
pub struct Blake3Cols<T> {
// 입력 데이터 (16개 32비트 워드, 각각 32비트로 표현)
```

```
pub inputs: [[T; 32]; 16],
// 체이닝 값들 (이전 압축의 출력)
pub chaining_values: [[[T; 32]; 4]; 2],
// 7라운드의 전체 라운드 데이터
pub full_rounds: [FullRound<T>; 7],
// 최종 출력
pub outputs: [[[T; 32]; 4]; 4],
}
```

제약조건 검증 과정:

1. **초기화 검증**: 체이닝 값과 상수의 올바른 설정 확인
2. **라운드 함수 검증**: 각 쿼터 라운드의 정확한 계산 확인
3. **비트 연산 검증**: XOR와 회전 연산의 정확성 확인
4. **최종 출력 검증**: 상태와 체이닝 값의 XOR 결과 확인

```
// 쿼터 라운드 함수 검증 예시
fn quarter_round_function<AB: AirBuilder>(
&self,
builder: &mut AB,
trace: &QuarterRound<AB::Var, AB::Expr>,
) {
// a' = a + b + m_{2i} mod 2^32 검증
add3(builder, trace.a_prime, trace.a, &[b_0_16, b_16_32], trace.m_two_i);
```

```
// d' = (a' ^ d) >> 16 검증 (XOR와 회전)

xor_32_shift(builder, trace.a_prime, trace.d, trace.d_prime, 16);

// 추가 연산들...

}
```

2. Keccak AIR 구현

Keccak은 5×5 상태 배열에서 동작하는 순열 기반 해시 함수입니다.

주요 특징:

- **상태 표현**: 5×5 배열의 64비트 워드들
- **라운드 구조**: 24라운드의 θ, ρ, π, χ, ι 단계
- **비트 레벨 연산**: 대부분의 연산이 비트 레벨에서 수행

```
pub struct KeccakCols<T> {

// 라운드 플래그 (현재 라운드 표시)

pub step_flags: [T; NUM_ROUNDS],

// 순열 입력 (y-major 순서로 저장)

pub preimage: [[[T; U64_LIMBS]; 5]; 5],

// 5×5 상태 배열

pub a: [[[T; U64_LIMBS]; 5]; 5],

// θ 단계의 중간 값들

pub c: [[T; 64]; 5],

pub c_prime: [[T; 64]; 5],

// 각 단계의 출력들
```

```
pub a_prime: [[[T; 64]; 5]; 5],
pub a_prime_prime: [[[T; U64_LIMBS]; 5]; 5],
}
```

제약조건 검증:

1. **라운드 플래그 검증**: 정확히 하나의 라운드만 활성화
2. **θ 단계**: 컬럼별 XOR와 회전 연산 검증
3. **ρ와 π 단계**: 비트 회전과 위치 재배열 검증
4. **χ 단계**: 비선형 변환 검증
5. **ι 단계**: 라운드 상수 추가 검증

```
// θ 단계 검증 예시
for x in 0..5 {
builder.assert_bools(local.c[x].clone());
builder.assert_zeros::<64, _>(array::from_fn(|z| {
let xor = local.c[x][z].clone().into().xor3(
&local.c[(x + 4) % 5][z].clone().into(),
&local.c[(x + 1) % 5][(z + 63) % 64].clone().into(),
);
local.c_prime[x][z].clone() - xor
}));
}
```

3. Poseidon2 AIR 구현

Poseidon2는 필드 원소에서 직접 동작하는 해시 함수로, 다른 해시 함수들과 달리 비트 연산이 필요하지 않습니다.

주요 특징:

- **필드 네이티브**: 유한체 원소에서 직접 연산
- **S-box 기반**: 거듭제곱 연산을 통한 비선형성
- **라운드 구조**: 전체 라운드와 부분 라운드의 조합

```
pub struct Poseidon2Cols<T, const WIDTH: usize, const SBOX_DEGREE: u64,
const SBOX_REGISTERS: usize, const HALF_FULL_ROUNDS: usize,
const PARTIAL_ROUNDS: usize> {
pub inputs: [T; WIDTH],
// 시작 전체 라운드들
pub beginning_full_rounds: [FullRound<T, WIDTH, SBOX_DEGREE, SBOX_REGISTERS>; H
// 부분 라운드들 (첫 번째 원소만 S-box 적용)
pub partial_rounds: [PartialRound<T, WIDTH, SBOX_DEGREE, SBOX_REGISTERS>; PARTI
// 끝 전체 라운드들
pub ending_full_rounds: [FullRound<T, WIDTH, SBOX_DEGREE, SBOX_REGISTERS>; HALF
}
```

S-box 최적화:

Poseidon2는 S-box 계산을 최적화하기 위해 중간 결과를 저장합니다:

```
pub struct SBox<T, const DEGREE: u64, const REGISTERS: usize>(pub [T; REGISTERS
```

```
// 예: x^7 = x * x^2 * x^4 (3개의 중간 결과 필요)
// REGISTERS = 3: [x^2, x^4, x^6] 저장
```

AIR 유틸리티 함수들

Plonky3는 AIR 구현을 위한 다양한 유틸리티 함수를 제공합니다:

1. 비트 패킹과 언패킹

```
// 비트들을 리틀 엔디안으로 패킹
pub fn pack_bits_le<R, Var, I>(iter: I) -> R
where
R: PrimeCharacteristicRing,
Var: Into<R> + Clone,
I: DoubleEndedIterator<Item = Var>;

// 32비트 정수를 비트 배열로 변환
pub fn u32_to_bits_le<R: PrimeCharacteristicRing>(val: u32) -> [R; 32];
```

2. 모듈러 산술 검증

```
// a = b + c + d mod 2^32 검증
pub fn add3<AB: AirBuilder>(
builder: &mut AB,
a: &[AB::Var; 2], // 결과 (16비트 limb 2개)
b: &[AB::Var; 2], // 첫 번째 피연산자
c: &[AB::Expr; 2], // 두 번째 피연산자
d: &[AB::Expr; 2], // 세 번째 피연산자
);
```

3. XOR와 시프트 연산

```
// a = (b ^ (c << shift)) 검증
pub fn xor_32_shift<AB: AirBuilder>(
builder: &mut AB,
a: &[AB::Var; 2], // 결과 (16비트 limb 2개)
b: &[AB::Var; 32], // 첫 번째 피연산자 (비트 배열)
c: &[AB::Var; 32], // 두 번째 피연산자 (비트 배열)
shift: usize, // 시프트 양
);
```

STARK 증명 시스템과의 통합

1. 심볼릭 빌더

AIR 제약조건은 먼저 심볼릭 표현으로 변환됩니다:

```
pub struct SymbolicAirBuilder<F: Field> {
// 심볼릭 변수들
main: RowMajorMatrix<SymbolicVariable<F>>,
// 제약조건 수집
constraints: Vec<SymbolicExpression<F>>,
}
```

2. 증명자 폴더

실제 증명 생성 시에는 구체적인 값들로 제약조건을 평가합니다:

```
pub struct ProverConstraintFolder<'a, SC: StarkGenericConfig> {
// 실제 추적 데이터
main: Matrix<PackedVal<SC>>,
// 제약조건 누적
accumulator: PackedChallenge<SC>,
}
```

3. 검증자 폴더

검증 시에는 커밋된 값들을 사용하여 제약조건을 확인합니다:

```
pub struct VerifierConstraintFolder<'a, SC: StarkGenericConfig> {
// 커밋된 값들
```

```
main: Vec<SC::Challenge>,

// 검증 결과

result: SC::Challenge,

}
```

성능 최적화 기법

1. 패킹된 필드 연산

여러 필드 원소를 SIMD 레지스터에 패킹하여 병렬 처리:

```
type PackedVal<SC> = <Val<SC> as Field>::Packing;


// 8개 또는 16개 필드 원소를 동시에 처리

impl PackedValue for PackedBabyBear {

const WIDTH: usize = 16; // AVX-512의 경우

}
```

2. 조건부 병렬화

`maybe-rayon` 을 통한 선택적 병렬 처리:

```
use p3_maybe_rayon::prelude::*;


// 병렬 처리 가능한 경우 자동으로 병렬화
```

```
constraints.par_iter().for_each(|constraint| {
// 제약조건 평가
});
```

3. 메모리 레이아웃 최적화

구조체의 메모리 레이아웃을 최적화하여 캐시 효율성 향상:

```
#[repr(C)]
pub struct Blake3Cols<T> {
// 자주 접근되는 데이터를 앞쪽에 배치
pub inputs: [[T; 32]; 16],
// ...
}
```

보안 고려사항

1. 제약조건 완전성

모든 계산 단계가 적절한 제약조건으로 보호되어야 합니다:

- **입력 검증**: 모든 입력이 유효한 범위 내에 있는지 확인
- **중간 상태 검증**: 각 계산 단계의 정확성 보장
- **출력 검증**: 최종 결과의 일관성 확인

2. 사운드니스 분석

제약조건의 차수와 개수가 보안 매개변수에 미치는 영향:

```
// 제약조건 차수 계산
let log_quotient_degree = get_log_quotient_degree::<Val<SC>, A>(
air,
0,
public_values.len(),
config.is_zk()
);
```

3. 영지식 속성

필요한 경우 추가 랜덤성을 통한 영지식 속성 보장:

```
// ZK가 활성화된 경우 추적 확장
let ext_trace_domain = pcs.natural_domain_for_degree(
degree * (config.is_zk() + 1)
);
```

실제 사용 예제

1. 간단한 피보나치 AIR

```
struct FibonacciAir;
```

```
impl<AB: AirBuilder> Air<AB> for FibonacciAir {
fn  eval(&self, builder: &mut AB) {
let  main = builder.main();
let  local = main.row_slice(0);
let  next = main.row_slice(1);
// 첫 번째 행: a = 1, b = 1
builder.when_first_row().assert_one(local[0]);
builder.when_first_row().assert_one(local[1]);
// 전환 행: next_a = local_b, next_b = local_a + local_b
builder.when_transition().assert_eq(next[0], local[1]);
builder.when_transition().assert_eq(next[1], local[0] + local[1]);
}
}
```

2. 해시 체인 검증

```
struct HashChainAir<H: HashFunction> {
hash_function: H,
}

impl<AB: AirBuilder, H: HashFunction> Air<AB> for HashChainAir<H> {
fn  eval(&self, builder: &mut AB) {
let  main = builder.main();
```

```
let local = main.row_slice(0);

let next = main.row_slice(1);

// 각 행에서 해시 함수 적용 검증

builder.when_transition().assert_eq(

next.hash_output(),

self.hash_function.eval(builder, local.hash_input())

);

}

}
```

결론

AIR 시스템은 Plonky3의 핵심 구성 요소로, 다양한 계산을 STARK 증명 시스템에서 검증 가능한 형태로 변환하는 강력한 추상화를 제공합니다. 각 해시 함수의 AIR 구현은 해당 함수의 특성에 맞게 최적화되어 있으며, 공통된 유틸리티 함수들을 통해 효율적인 구현이 가능합니다.

주요 장점:

- **모듈성**: 다양한 계산을 통합된 방식으로 처리
- **효율성**: 패킹된 연산과 병렬 처리를 통한 성능 최적화
- **안전성**: 완전한 제약조건을 통한 계산 정확성 보장
- **확장성**: 새로운 계산 유형을 쉽게 추가 가능

AIR 시스템의 이해는 Plonky3를 활용한 고성능 영지식 증명 시스템 구축의 핵심이며, 각 구성 요소의 역할과 상호작용을 파악하는 것이 중요합니다.